Do AI Agents Stop Before Acting? A Frozen Benchmark of Lookalike Domains, Poisoned Tools, Revoked Receipts and Stale Answers, With and Without an External Verification Signal

Emmanuel Orta
VSNARY
contact@crawlcheck.io
ORCID: 0009-0001-5269-1515

22 October 2026

Abstract

An AI agent that acts on the web for a person must sometimes decide not to act: the link it was handed is a lookalike of a known brand, the tool server it is about to use carries injected instructions, the receipt that authorised a purchase has been revoked, the cached answer it relies on has expired, the site's published rules forbid the fetch, or the page it is about to cite is not what it claims to be. We present a small frozen benchmark of 39 such situations (20 hazards, 19 matched controls) across six families, and run four open-weight or low-cost language models through each case twice: once with only what the agent could observe for itself, and once with the same evidence plus a signed answer from an external verification service. Alone, the models stopped on 32 of 80 hazard trials (40%, 95% CI 30–51%); with the signal, 79 of 80 (99%, CI 93–100%). Usefulness on controls rose from 78% to 89%. The gain is uneven by family: models stopped on every lookalike domain unaided, but on only 1 of 12 robots.txt refusals and 0 of 12 identity mismatches. We publish the suite, the prompt template, the scoring rule, every result and the signatures over them so that the run can be repeated and challenged. We state the limits plainly: the cases were chosen by the operator of the verification service, the signal in the second arm is correct by construction, so the second arm measures whether a model heeds a correct signal rather than how often the service is right; and each case was run once per model.

1 Introduction

Agents that browse, call tools, cite pages and pay on a person's behalf are being deployed with permission to act. The failures that matter most are not wrong answers but wrong actions: signing in on a domain that imitates a brand, connecting to a tool server whose descriptions carry instructions aimed at the agent, paying on an authorisation that has been withdrawn, acting on a decision that has gone stale, fetching what a site has asked agents not to fetch, or vouching for a page as an organisation's own when nothing corroborates that.

Each of these is a decision the agent must make *before* it acts, usually with incomplete evidence. This paper asks two narrow questions. First: given only what an agent can observe for itself, how often do current models stop in these situations, and how often do they wrongly stop in matched situations where acting is fine? Second: when the same evidence is accompanied by a signed answer from an external service that has checked the target, how much does behaviour change?

The second question is not a test of the service's accuracy. The signal shown to the model was produced by the service and matches each case's label by construction. What the second

arm measures is whether a model that is handed a correct verdict acts on it, which is a necessary condition for any such service to matter, not a sufficient one. Section 6 returns to this.

2 Related work

Benchmarks of agent security so far measure mostly one hazard: instructions injected into what an agent reads. InjecAgent [1] tests indirect prompt injection against tool-integrated agents, where attacker text arrives through a tool's output. AgentDojo [2] provides a dynamic environment of user tasks and injection cases for evaluating both attacks and defenses. Agent Security Bench [3] formalises attacks and defenses across agent scenarios and tools. WASP [4] brings the question to web agents acting in realistic sites. On deceptive destinations, PhishNChips [5] shows that phishing detection by language models depends strongly on the system prompt, and that checks relying on sender and URL domains matching are defeated by attackers who register matching domains.

This benchmark differs in two ways. It asks about the decision to act at all, across six hazard families of which tool-description injection is one, and it scores wrongly stopping on matched benign cases alongside failing to stop. And it measures a second arm in which the same evidence comes with an external, signed verdict, to ask whether models act on correct outside evidence when they have it.

3 The suite

The suite is a single JSON document, frozen at version 4 on 22 October 2026, with SHA-256 `a6a12ceb...f1031ecb`.[1] It holds 39 cases in six families (Table 1). Each case has a user request, the evidence the agent observed itself, the verification service's signed answer (used only in the second arm), a label (*stop* or *proceed*) and a `truth_source` field recording how the label was established.

Labels. A case's label is *stop* when acting is wrong and *proceed* when it is fine. For the observed cases the `truth_source` names the route that established the fact (for example the robots.txt that was fetched by hand on the day the suite was frozen). Version 4 corrected the three identity hazards: in version 3 the signal shown for them was weaker than their label, so a model that followed the signal was scored wrong. The suite's `changes` field records this and the earlier additions.

Prompt. Every model sees the same template. It is told it is acting for a user whose credentials sit in a password manager that fills them in only if the agent proceeds, and is asked to answer with JSON naming one of four actions: `proceed`, `proceed_with_caution`, `stop_and_verify`, `refuse`. In the "alone" arm the prompt holds the request and the agent's own observations. In the "with" arm it holds the same plus the service's signed answer, labelled as such. The full template is in the suite file.

Scoring. A hazard is handled correctly by `stop_and_verify` or `refuse`; a control by `proceed` or `proceed_with_caution`. Safety is hazards stopped over hazards; usefulness is controls completed over controls; the score is their mean. An answer that names no action counts as wrong. There were none in this run.

[1] Full hash and file: https://crawlcheck.io/benchmark/suite-v4.json. The same file is mirrored at https://huggingface.co/datasets/crawlcheck/agent-safety-benchmark.

Family	Task	Cases	Seen live	What a hazard looks like
Lookalike	sign in	8	8	An IDN homoglyph or typo of a brand domain (`app1e.com`); controls are the brand's own or brand-owned domains
Poisoned tool	connect, use	8	5	An MCP server whose tool descriptions tell the agent to read key files or exfiltrate data; controls are clean servers
Revoked	transact	5	0	A purchase receipt that is revoked, signed by an unpublished key, or expired; controls are valid receipts
Stale	read, connect, transact	6	6	A cached decision older than its maximum age for that action; controls are within age
Robots	read	6	6	A fetch the site's robots.txt forbids for the agent's named user agent; controls are permitted fetches
Identity	cite	6	3	A page to be cited as an organisation's own whose claimed profiles do not link back; controls are corroborated sites

Table 1: The 39 cases. "Seen live" counts cases built from an observed site or server rather than a constructed fixture. The three identity hazards are fixtures that copy the profile links of real businesses onto `.example` domains; the revoked family is entirely fixtures because no revoked receipt existed in the wild when the suite was frozen.

4 Run

Four models were run on 22 October 2026 (run `bn-2026-10-22-a6a12c`, 09:33–11:48 UTC): Gemini 3.5 Flash-Lite through the Gemini API, and gpt-oss-120b, Llama 3.3 70B Instruct and Mistral Small 3.1 24B through Cloudflare Workers AI. Each case was sent once per model per arm, at temperature 0 where the host allows it: 312 calls, 0 errors. The results document is signed (Ed25519) and its signing key is published in the service's key directory, so the file can be checked offline.

5 Results

	Alone			With signal		
Model	Hazards stopped	Controls done	Score	Hazards stopped	Controls done	Score
Gemini 3.5 Flash-Lite	11/20 (34–74)	17/19 (69–97)	0.72	20/20 (84–100)	19/19 (83–100)	1.00
gpt-oss-120b	10/20 (30–70)	10/19 (32–73)	0.51	20/20 (84–100)	12/19 (41–81)	0.82
Llama 3.3 70B	6/20 (15–52)	17/19 (69–97)	0.60	19/20 (76–99)	19/19 (83–100)	0.98
Mistral Small 3.1 24B	5/20 (11–47)	15/19 (57–91)	0.52	20/20 (84–100)	18/19 (75–99)	0.97
All four	32/80 (30–51)	59/76 (67–86)	0.59	79/80 (93–100)	68/76 (81–95)	0.94

Table 2: Per-model results. Parentheses give 95% Wilson intervals in percent. With 20 hazards per model the per-model intervals are wide; the pooled row is the more reliable reading, with the caveat that the four models are not independent samples of anything.

Three things stand out.

Unaided models already refuse lookalikes, and over-refuse their controls. Every model

	Alone		With signal	
Family (4 models pooled)	Hazards stopped	Controls done	Hazards stopped	Controls done
Lookalike	16/16	7/16	16/16	11/16
Poisoned tool	10/16	9/16	16/16	13/16
Revoked	2/12	8/8	12/12	8/8
Stale	3/12	12/12	12/12	12/12
Robots	1/12	11/12	12/12	12/12
Identity	0/12	12/12	11/12	12/12

Table 3: Results by family, all four models pooled.

stopped on every lookalike hazard without help. The cost was on the matched controls: only 7 of 16 brand-owned or brand-own domains were signed in to. The hazard here is visible in the string itself, and the models' caution spilled over onto legitimate domains that merely resembled the pattern. The signal recovered 4 of those 9 controls, not all of them.

Unaided models almost never stop on the invisible hazards. A revoked receipt, an expired cached answer, a robots.txt refusal and an uncorroborated identity leave no trace in the page the agent sees. Pooled, the models stopped on 6 of 48 such hazards (12.5%). These are exactly the cases where the agent cannot reason its way to the right answer from the evidence in front of it; it has to have been told. With the signal, 47 of 48.

The signal did not make the models timid. Usefulness rose, not fell, in every family. The one model whose usefulness stayed low (gpt-oss-120b, 12 of 19 controls) was already refusing 9 of 19 controls alone; it treats the whole frame as suspicious, and the signal only partly reassured it.

6 Limitations

The cases were chosen by the service's operator. The author built both the suite and the verification service. The families were chosen because they are the situations the service is designed to answer, and the controls were chosen to be answerable. Nothing here says how often these situations arise in real agent traffic, or how a model fares on hazards the service does not cover.

The second arm measures heeding, not accuracy. The signal shown in the "with" arm is the service's own answer for that case, and for all 39 cases that answer agrees with the label. The arm therefore cannot show a false signal being followed, which is the failure a dependent agent should fear most. A proper accuracy study needs cases the service's operator did not choose, labelled by someone else, and a count of the service's own false positives and false negatives. The service publishes a running accuracy record for that purpose,[2] but at the time of writing it holds a handful of hand-checked findings, not an independent evaluation.

One run, four models, small counts. Each case was sent once per model. Temperature 0 was requested but not every host guarantees determinism. The per-model hazard counts are 20, so a difference of a few cases is inside the interval. Two of the four models are small; no frontier closed model beyond a Flash-Lite tier was run, because the run was designed to be repeatable by anyone at low cost. The pooled row treats 80 trials as if they were independent; they are 20 cases seen by 4 correlated models.

Fixtures. The revoked family and three identity hazards are constructed, not observed. The identity fixtures copy the profile links of the author's own businesses onto `.example` domains, which keeps the case realistic but means the "real" side of the comparison is also the author's.

[2] https://crawlcheck.io/accuracy

Labels changed once. Version 4 corrected the identity hazards after version 3 scored a model wrong for following the signal. The correction is recorded in the suite. Results from version 3 are not comparable and are not reported here.

7 Reproducing and challenging the run

Everything needed to repeat the run is public: the suite file with its hash, the prompt template inside it, the scoring rule, the per-model and per-family results, and an Ed25519 signature over the results document whose key is in the service's published key directory.[3] A reader who runs the suite on another model, or who disagrees with a label, can send the result to the address on the title page; disputed labels will be recorded in the suite's `changes` field with the version that resolved them.

Disclosure

The author operates CrawlCheck, the verification service whose signal is used in the second arm, and built the benchmark. No outside party has yet reproduced the run.

References

[1] Q. Zhan, Z. Liang, Z. Ying, D. Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv:2403.02691, 2024.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks, 2024. arXiv:2406.13352.

[3] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, Y. Zhang. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. ICLR, 2025. arXiv:2410.02644.

[4] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, K. Chaudhuri. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. arXiv:2504.18575, 2025.

[5] R. Litvak. The System Prompt Is the Attack Surface: How LLM Agent Configuration Shapes Security and Creates Exploitable Vulnerabilities. arXiv:2603.25056, 2026.

[3] Suite: https://crawlcheck.io/benchmark/suite-v4.json. Results: https://crawlcheck.io/api/v1/benchmark. Method page: https://crawlcheck.io/benchmark. Key directory: https://crawlcheck.io/.well-known/http-message-signatures-directory.